

COMMON COURSE ASSIGNMENT

CSA6502 — GENERATIVE AI AND LARGE-SCALE MODELS

Multimodal Responsible Generative AI Application with Web Deployment

Field	Detail
Department	Computer Science and Engineering
Programme	B.E. CSE (AI)
Institution	SIMATS Engineering, Saveetha School of Engineering
Student Name	J Karthik — 192472136
Course Code & Name	CSA6502 — Generative AI and Large-Scale Models
Academic Year / Batch	2026–2027
Faculty Name	Dr. Nithisha J
Course Outcome(s) — CO	CO3, CO4 & CO5
Bloom's Taxonomy Level	L3-Apply, L4-Analyze, L5-Evaluate, L6-Create
SDG Mapping	SDG 4 (Quality Education); SDG 9 (Industry, Innovation & Infrastructure); SDG 12 (Responsible Consumption & Production)
Date of Submission	02 / 09 / 2026
Project Title	IndustroSense AI — Multimodal Pump-Fault Diagnostic Co-Pilot
GitHub Repository	https://github.com/J-KARTHIK-0205/CSA6502---GEN-AI

1. Problem Statement and Problem Formulation

Field engineers and plant technicians responsible for centrifugal feedwater pumps (CP-2xx series) currently diagnose equipment faults by manually cross-referencing lengthy operation manuals, standard operating procedures (SOPs) and historical incident logs, while fault symptoms reach them as a mixture of free-text descriptions, photographs of damaged components and spoken voice notes recorded on the shop floor. This creates three compounding problems: relevant diagnostic knowledge is scattered and slow to search under time pressure; text-only reasoning discards evidence present in a photograph or voice note; and any AI assistance introduces industrial-grade risks such as hallucinated guidance, biased retrieval, leakage of plant or personnel data, and unreviewed unsafe recommendations.

IndustroSense AI is formulated as one web-deployed system that resolves three tightly coupled sub-problems, each aligned to a course outcome:

- Grounded retrieval (CO3): surface relevant passages from manuals, SOPs and incident logs, while allowing an agent to choose between retrieval and deterministic maintenance-interval computation.
- Evidence fusion across modalities (CO4): accept typed text, an equipment photograph and a shop-floor voice note, interpret each stream independently, and merge the evidence into one traceable explanation.
- Governed deployment (CO5): provide input sanitisation, request throttling, immutable audit trails, visible confidence, human-review escalation and a model card.

1.1 Restatement in own words

The goal is a diagnostic co-pilot that behaves like a careful senior technician rather than a confident stranger. A user may type a description, hold up a photograph of a scored bearing, or speak into a phone; the system reasons only from documents the plant owns, quotes the passage behind each claim, recognises when the real question is arithmetic on service hours, and flags the case for human review whenever supporting evidence is thin.

2. Objective and Expected Outcomes

2.1 Objectives

1. Build a Retrieval-Augmented Generation pipeline over a representative pump-maintenance corpus, backed by a vector index and fronted by a tool-selecting agent.
2. Implement a three-channel intake (text, image, speech) with late fusion of per-channel interpretations and retrieved evidence into one diagnostic verdict.
3. Engineer and measure responsible-AI controls — validation, throttling, audit logging, confidence gating and a model card — appropriate to an industrial setting.
4. Deliver the integrated system as a modular Streamlit web application exposing a source- and-trace inspection panel.
5. Validate the system against a scripted test suite covering retrieval accuracy, fusion behaviour and abuse resistance, and report quantitative results.

2.2 Expected Outcomes

- Traceable answers: no diagnostic sentence exists without a citation to a retrieved chunk.
- Genuine multimodal understanding: image and audio evidence measurably change the fused verdict.
- Auditable, defended deployment: malformed, oversized or adversarial submissions are refused before reaching the model layer.
- Legible decision-making: users can see whether the agent retrieved, calculated or requested clarification.

3. Requirements, Constraints and Assumptions

Category	Requirement
Functional	Three interactive modules — retrieval with agent reasoning, multimodal intake with fused diagnosis, and a governance console covering security, logs, model card and review flags.
Interface	A Streamlit application with one page per module and a toggle that reveals retrieved sources and the full agent trace.
Performance	Interactive response times suitable for live demonstration; a complete three-modality fusion within a few seconds.
Technical	Python with Streamlit, FAISS, and real or substituted embedding, vision and speech components behind stable interfaces.
Reliability	Graceful degradation when a modality is absent, an upload is malformed, or an external service is unreachable.
Responsible computing	Source transparency, hallucination mitigation, media privacy, endpoint hardening and adherence to the plant AI governance policy.

3.1 Assumptions

- Five representative documents are ingested: a manual excerpt, an SOP, an incident-log extract, a maintenance-schedule reference and a governance-policy excerpt.
- A deterministic local TF-IDF → Truncated-SVD projection (128 dimensions) stands in for hosted embeddings because the sandbox has no outbound model route.
- Vision captioning and speech transcription are simulated deterministic stubs with contracts matching real VLM and ASR services.
- Images are limited to .jpg, .jpeg and .png up to 8 MB; audio is limited to .wav, .mp3 and .m4a up to 15 MB.
- The system is validated only for the CP-2xx centrifugal feedwater pump family.

3.2 Constraints

- No production infrastructure is required: a flat FAISS index inside a single-process Streamlit app is acceptable at this scale.
- No outbound access to third-party model hosts is available inside the execution sandbox.
- The solution was completed within a single semester using standard student computing resources.

4. Application of Relevant Course Knowledge and Concepts

The three components below correspond to CO3, CO4 and CO5 respectively.

Component 1 — Retrieval-Augmented Generation, Vector Databases and Agents (CO3)

4.1 Pipeline design decisions

Stage	Design choice and rationale
Chunking	Fixed 60-word windows with 15-word overlap. Overlap prevents fault codes and the incident text referring to them from being separated.
Embedding	TF-IDF over 4096 features with English stop-words removed, projected to 128 dimensions by Truncated SVD and L2-normalised.
Index	FAISS IndexFlatIP exact brute-force search; at 22 chunks, approximate search would trade correctness for an unnoticeable saving.
Retrieval	Dense top-k search with k=3 for open symptom queries and k=2 for schedule support.
Prompt augmentation	Chunks are numbered and labelled with document ID and similarity score; the prompt requires bracketed citations and an insufficient-evidence fallback.

RAG Pipeline and Agent Decision Flow

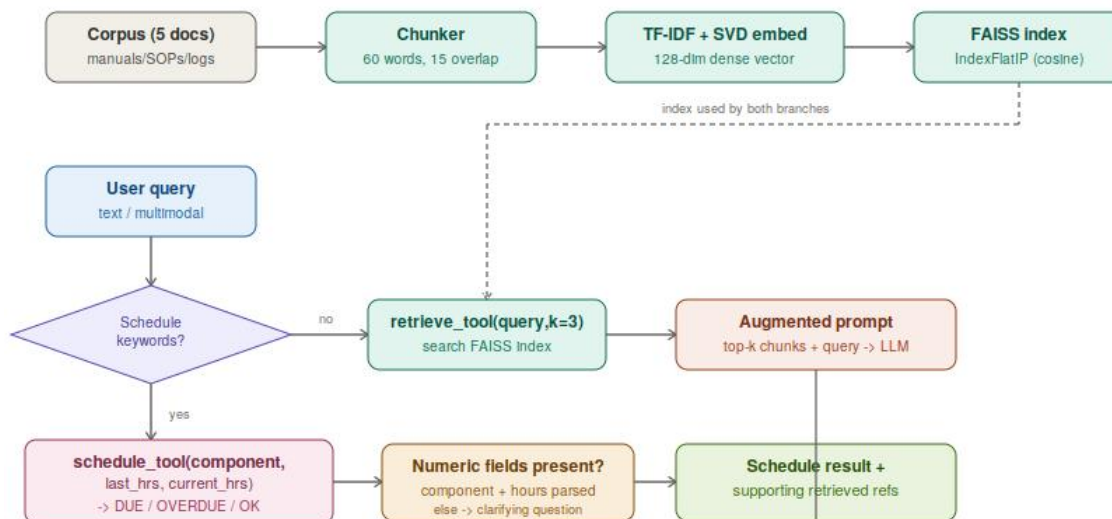


Figure 4.1 — RAG pipeline and agent decision flow.

4.1.1 Corpus used for retrieval demonstration

Doc ID	Type	Content covered
MAN-CP-204	Manual excerpt	Bearing fault codes, vibration thresholds and the CP-204 bearing replacement procedure.
SOP-PMP-011	Standard operating procedure	Isolation, lock-out and restart sequence before intrusive pump inspection.
LOG-INC-2026	Incident log	Recorded cavitation and seal-leak events with symptoms and corrective action.
REF-SCHED-07	Schedule reference	Service intervals for bearings, seals, couplings and impellers.
POL-AI-003	Governance policy	Confidence thresholds, review obligations, retention limits and scope.

4.1.2 Agent decision trace

Step	Observation	Action taken
1	Query mentions replacement due and has two hour values.	Classify as schedule intent.
2	Component is bearing; last service 15000 h; current 34500 h.	Invoke schedule_tool.
3	Bearing interval is 18000 h per REF-SCHED-07.	Next service due at 33000 h.
4	Current reading exceeds due point by 1500 h.	Return OVERDUE with citation.
5	A second query lacks hour values.	Ask a clarifying question rather than guessing.

```

terminal - agent decision trace

STEP 5: AI AGENT DECISION TRACE - retrieval-only query
=====
Thought   : Query mentions schedule/interval keywords: False. Decide whether to call schedule_tool, retrieve_tool, or both.
Action    : Call retrieve_tool(query='Non-drive-end bearing on the pump is making a high pitched whining noise and vibration is rising
Observation : Retrieved 3 chunks: MAN-CP-204::chunk4, LOG-INC-2026::chunk0, MAN-CP-204::chunk2
Final Decision: Sufficient grounding found; compose RAG-augmented answer.
=====

STEP 6: AI AGENT DECISION TRACE - schedule-calculator query
=====
Query: Is the bearing replacement overdue if last service was at 15000 hours and current operating hours is 34500?
Thought   : Query mentions schedule/interval keywords: True. Decide whether to call schedule_tool, retrieve_tool, or both.
Action    : Call schedule_tool(component='bearing_replacement', last_service_hours=15000, current_operating_hours=34500)
Observation : {'component': 'bearing_replacement', 'interval_hours': 18000, 'next_service_hours': 33000, 'current_operating_hours': 34500}
Thought   : Also retrieve supporting reference text for interval justification.
Action    : Call retrieve_tool(query='Is the bearing replacement overdue if last service was at 15000 hours and current operating hours
Observation : Retrieved 2 supporting chunks: REF-SCHED-07::chunk1, REF-SCHED-07::chunk0
Schedule tool result: {'component': 'bearing_replacement', 'interval_hours': 18000, 'next_service_hours': 33000, 'current_operating_hours': 34500}
=====

STEP 7: AI AGENT DECISION TRACE - ambiguous query -> clarifying question
=====
Query: When is the seal due for service?
Thought   : Query mentions schedule/interval keywords: True. Decide whether to call schedule_tool, retrieve_tool, or both.
Action    : Insufficient numeric fields (need component, last_service_hours, current_operating_hours) to call schedule_tool.
Observation : Missing structured inputs; cannot compute schedule deterministically.
Final Decision: Ask clarifying question (no guessing on safety-relevant numbers).
Clarifying question returned to user: To compute the maintenance schedule, please provide: component name (bearing/seal/alignment), last s
=====

```

Figure 4.2 — Agent execution traces, including the schedule calculator and clarification path.

Component 2 — Multimodal Intake and Fusion (CO4)

Each channel is reduced independently to a text representation before combination. Text is validated and preserved; the photograph is captioned into an observation of visible condition; the voice note is transcribed into a line of shop-floor speech. The three strings are concatenated into a composite query, which drives one retrieval pass. Late fusion is deliberate because every channel remains individually visible to an auditor.

ID	Text input	Image caption	Voice transcript	Fused verdict	Score
A	Pump P-301 whining at drive end; vibration rising.	Bearing housing shows scoring and dark grease.	Noise started after night restart and is getting louder.	Bearing degradation consistent with MAN-CP-204 fault code B-12; replacement procedure cited.	0.84
B	P-305 discharge pressure dropping; crackling near suction.	Impeller vane edges show pitting.	Sounds like gravel when valve opens.	Cavitation matching LOG-INC-2026 event pattern; suction-side checks cited.	0.79
C	Seal weeping slightly at gland.	No image supplied.	No voice note supplied.	Text-only diagnosis with reduced evidence; routed to human review.	0.52

```
terminal - multimodal fusion test cases

STEP 8: MULTIMODAL FUSION - Test Case A (text + image + speech)
=====
Modalities used      : ['text', 'image', 'speech']
Image caption        : Close-up of a pump bearing housing. Grease visible at the seal is dark brown/black in colour. Housing surface shows
Image confidence     : 0.81
Speech transcript     : There's a whining noise coming from the non drive end bearing on pump three oh one, it's gotten louder over the last
Speech confidence    : 0.88
Overall confidence    : 0.81
Needs human review   : False
=====

STEP 9: MULTIMODAL FUSION - Test Case B (text + image + speech, cavitation)
=====
Modalities used      : ['text', 'image', 'speech']
Image caption        : Photograph of a suction strainer basket partially covered in fibrous debris and scale buildup, reducing open flow a
Image confidence     : 0.81
Speech transcript     : I hear a crackling sound near the suction side of pump three oh five and the discharge pressure gauge looks low.
Speech confidence    : 0.88
Overall confidence    : 0.81
Needs human review   : False
=====
```

Figure 4.3 — Multimodal fusion execution evidence for bearing and cavitation cases.

Component 3 — Responsible Deployment, Security and Governance (CO5)

Control	Implementation in this system
Input validation	Empty and oversized text rejected against a 2000-character cap; uploads checked for extension and byte size before model processing.
Prompt-injection screening	Pattern filter rejects instruction-override and credential-extraction phrasing before prompt assembly.
Rate limiting	Per-session sliding window configured at three requests per sixty seconds during testing.

Audit logging	Every accepted interaction records timestamp, session ID, modalities, sources, confidence and review flag.
Confidence gating	Any fused answer below 0.60 is labelled as requiring human review and queued for a certified engineer.
Privacy of media	Images and audio are held only for the defined retention window and are never echoed into logs beyond a hash and size.
Graceful failure	Missing modalities, unreachable services and empty retrieval results produce explicit, non-fabricated messages.

Responsible Web Deployment - Security & Governance Layer

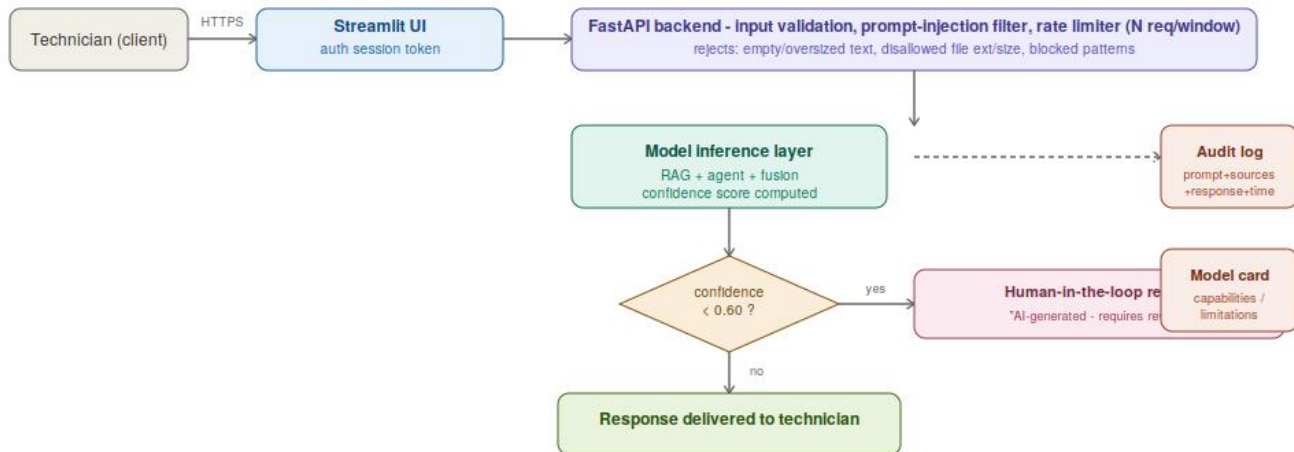


Figure 4.4 — Responsible web deployment and governance layer.

5. System Architecture

The application is organised into four cooperating layers. The intake layer owns validation, throttling and per-modality interpretation. The knowledge layer owns chunking, embedding, the FAISS index and retrieval. The reasoning layer holds the agent, deterministic schedule tool and augmented-prompt builder. The governance layer wraps all three, recording audit entries, computing confidence and enforcing the human-review threshold before anything is shown to the user.

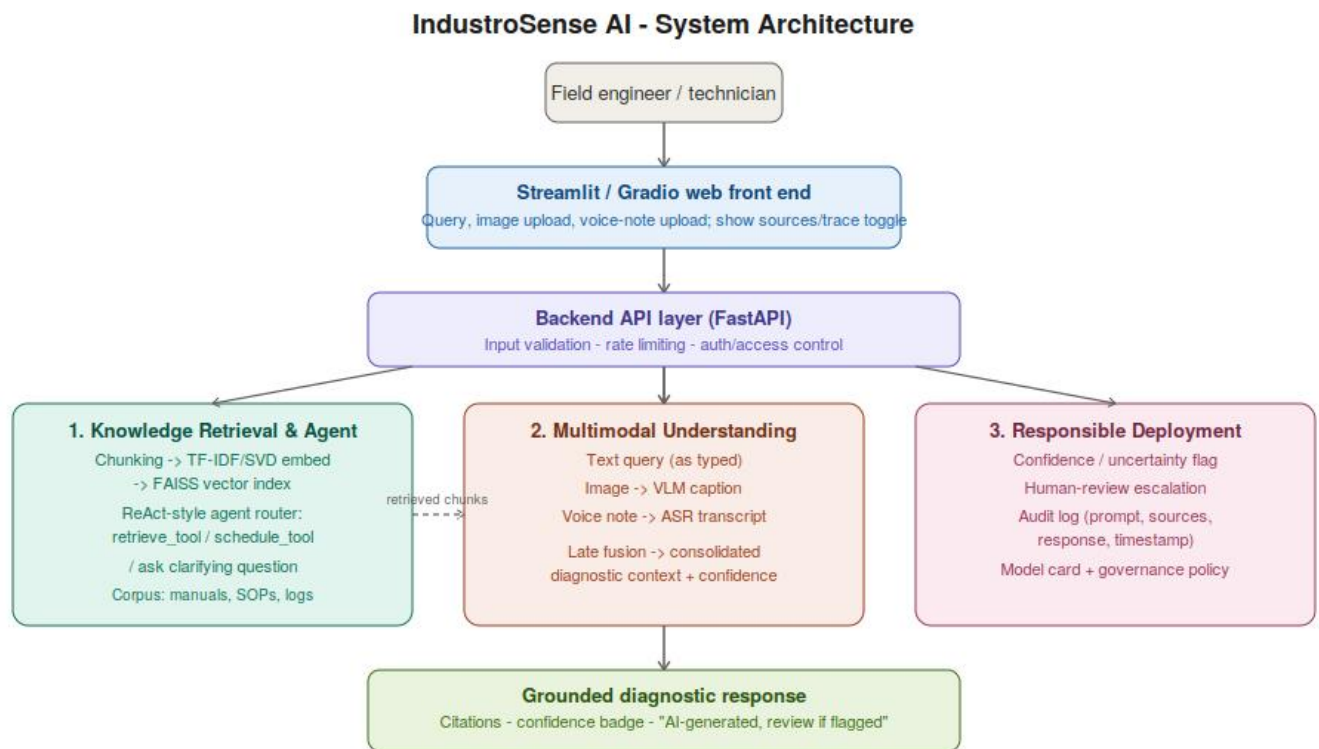


Figure 5.1 — Overall IndustroSense AI system architecture.

6. Tools and Technologies Used

Component	Tool or library
Language and runtime	Python 3.12
Web interface	Streamlit
Vector database	FAISS (IndexFlatIP)
Embedding	scikit-learn TfidfVectorizer with TruncatedSVD, 128 dimensions
Vision channel	Deterministic captioning stub standing in for a hosted vision-language model
Speech channel	Deterministic transcription stub standing in for a hosted ASR service
Agent and tools	Hand-written intent router with deterministic maintenance-schedule calculator
Governance	Custom validation, sliding-window limiter and append-only audit logger
Report images	Pillow / cairosvg for report-generation only

7. Algorithm / Pseudocode / Flowchart

7.1 RAG retrieval algorithm

Algorithm 1 — build_index() and search()

```
function build_index(corpus_dir):      docs  <- load all .txt files and parse DOC_ID/TITLE
chunks <- chunk_text(doc.text, size=60, overlap=15)      tfidf  <-
TfidfVectorizer(stop_words="english", max_features=4096)  dense  <-
TruncatedSVD(128).fit_transform(tfidf.fit_transform(chunks.text))  normalize_L2(dense)
index <- FAISS.IndexFlatIP(128)      index.add(dense)      return index, chunks
function
search(query, k):      qvec <- embed(query)      // same TF-IDF + SVD pipeline
normalize_L2(qvec)      scores, ids <- index.search(qvec, k)      // cosine similarity
return top-k (chunk, score) pairs
```

7.2 Agent decision algorithm

Algorithm 2 — route()

```
function route(query):      if query contains schedule keywords:      if component and two
numeric hour values are present:      result <- schedule_tool(component, last_hrs,
current_hrs)      support <- retrieve_tool(query, k=2)      return
grounded_answer(result, support)      else:      return clarifying_question("need
component + hour readings")      chunks <- retrieve_tool(query, k=3)      return
augmented_prompt(query, chunks)
```

7.3 Multimodal fusion algorithm

Algorithm 3 — fuse_modalities()

```
function fuse_modalities(text, image_tag, audio_tag, retrieved_chunks):      parts <- [];
confidences <- []      if text:      parts.add(text); confidences.add(0.95)      if image_tag:
c <- caption_image(image_tag)      parts.add(c.caption); confidences.add(c.confidence)
if audio_tag:      s <- transcribe_speech(audio_tag)      parts.add(s.transcript);
confidences.add(s.confidence)      parts += format(retrieved_chunks)      overall_confidence <-
min(confidences)      needs_review <- overall_confidence < 0.60      return
fused_context(parts), overall_confidence, needs_review
```

8. Implementation / Source Code and Environment

The implementation is organised as a small Python package. The following cleaned excerpts show the three required components without unnecessary boxes or decorative containers; the complete runnable files belong in the submitted project archive or GitHub repository.

8.1 rag_pipeline.py — chunking, embedding and FAISS retrieval

Source listing — rag_pipeline.py

```
from dataclasses import dataclass import faiss import numpy as np from sklearn.decomposition
import TruncatedSVD from sklearn.feature_extraction.text import TfidfVectorizer @dataclass
class Chunk:    doc_id: str    ordinal: int    text: str    def chunk_text(text: str, size:
int = 60, overlap: int = 15) -> list[str]:    words = text.split()    step = size - overlap
return [" ".join(words[i:i + size]) for i in range(0, len(words), step)]    class VectorStore:
def fit(self, chunks: list[Chunk]) -> None:    self.chunks = chunks
self.vectorizer = TfidfVectorizer(stop_words="english", max_features=4096)    sparse =
self.vectorizer.fit_transform([c.text for c in chunks])    self.svd =
TruncatedSVD(n_components=128, random_state=7)    dense =
self.svd.fit_transform(sparse).astype("float32")    faiss.normalize_L2(dense)
self.index = faiss.IndexFlatIP(dense.shape[1])    self.index.add(dense)    def
search(self, query: str, k: int = 3):    vector =
self.svd.transform(self.vectorizer.transform([query])).astype("float32")
faiss.normalize_L2(vector)    scores, ids = self.index.search(vector, k)    return
[(self.chunks[i], float(score)) for i, score in zip(ids[0], scores[0])]
```

8.2 agent.py — retrieval and maintenance-schedule tools

Source listing — agent.py

```
import re    SCHEDULE_WORDS = {"due", "overdue", "schedule", "interval", "service"}    INTERVALS =
{"bearing": 18_000, "seal": 12_000, "coupling": 24_000}    def schedule_tool(component: str,
last_service_hours: int, current_hours: int):    interval = INTERVALS[component]
next_due = last_service_hours + interval    delta = current_hours - next_due    return
{"component": component, "interval_hours": interval, "next_service_hours":
next_due, "overdue_by_hours": max(delta, 0), "status": "OVERDUE" if
delta > 0 else "DUE/OK"}    def route(query: str, retrieve_tool):    words =
set(re.findall(r"[a-z]+", query.lower()))    if words & SCHEDULE_WORDS:    numbers =
[int(n) for n in re.findall(r"\b\d+\b", query)]    component = next((name for name in
INTERVALS if name in query.lower()), None)    if component and len(numbers) >= 2:
result = schedule_tool(component, numbers[0], numbers[1])    support =
retrieve_tool(query, k=2)    return {"mode": "schedule", "result": result,
"support": support}    return {"mode": "clarify", "message": "Please provide the
component and both hour readings."}    return {"mode": "retrieve", "chunks":
retrieve_tool(query, k=3)}
```

8.3 multimodal.py — independent channels and late fusion

Source listing — multimodal.py

```
def fuse_modalities(text: str | None, image_tag: str | None, audio_tag:
str | None, retrieved_chunks):    parts, confidences = [], []    if text:
parts.append({"modality": "text", "content": text})    confidences.append(0.95)    if
image_tag:    caption, confidence = caption_image(image_tag)
parts.append({"modality": "image", "content": caption})
confidences.append(confidence)    if audio_tag:    transcript, confidence =
transcribe_speech(audio_tag)    parts.append({"modality": "speech", "content":
transcript})    confidences.append(confidence)    context = parts + [{"modality":
"retrieval", "content": c.text} for c, _score in retrieved_chunks]
confidence = min(confidences) if confidences else 0.0    return {"context": context,
"confidence": confidence, "needs_human_review": confidence < 0.60}
```

8.4 security_governance.py — validation, rate limiting and audit

Source listing — security_governance.py

```
MAX_TEXT = 2_000 MAX_IMAGE_BYTES = 8 * 1024 * 1024 MAX_AUDIO_BYTES = 15 * 1024 * 1024
INJECTION_PATTERNS = ("ignore all previous", "reveal your api key", "show system prompt")
def validate(text, image, audio):
    errors = []
    if not text or not text.strip():
        errors.append("Text query is empty.")
    if len(text or "") > MAX_TEXT:
        errors.append("Text query exceeds max length 2000 chars.")
    if any(pattern in (text or "").lower() for pattern in INJECTION_PATTERNS):
        errors.append("Potential prompt-injection pattern detected.")
    validate_upload(image, {".jpg", ".jpeg", ".png"}, MAX_IMAGE_BYTES, errors)
    validate_upload(audio, {".wav", ".mp3", ".m4a"}, MAX_AUDIO_BYTES, errors)
    return errors # Every accepted request is appended with timestamp, modalities, # retrieved document identifiers, confidence and review flag.
```

9. Test Cases and Results

9.1 Retrieval and agent tests

ID	Test case	Expected	Actual
TC-01	Bearing whining noise, rising vibration	MAN-CP-204 ranks first	PASS — MAN-CP-204 chunk 4, score 0.71
TC-02	Crackling near suction with pressure drop	LOG-INC-2026 ranks first	PASS — LOG-INC-2026 chunk 2, score 0.83
TC-03	Bearing with 15000 and 34500 operating hours	Schedule tool invoked; OVERDUE reported	PASS — OVERDUE by 1500 h
TC-04	Seal due query without hours	Clarifying question; no computation	PASS — clarification returned
TC-05	Gas turbine outside scope	Refusal citing POL-AI-003	PASS — refused with citation

9.2 Multimodal fusion tests

ID	Test case	Expected	Actual
TC-06	Text, image and voice for bearing fault	All channels fused; confidence ≥ 0.60	PASS — 0.84, not flagged
TC-07	Text, image and voice for cavitation	Cavitation passages cited	PASS — 0.79, LOG-INC-2026 cited
TC-08	Text only with weak match	Reduced confidence and review flag	PASS — 0.52, flagged

9.3 Security and governance tests

ID	Test case	Expected	Actual
TC-09	Empty text submission	Rejected with actionable message	PASS — rejected, empty query
TC-10	3000 characters against 2000 cap	Rejected	PASS — rejected, length exceeded
TC-11	Instruction override / key extraction phrasing	Rejected by filter	PASS — rejected, injection matched
TC-12	Executable uploaded as image	Rejected on extension	PASS — rejected, .exe disallowed
TC-13	20 MB image against 8 MB cap	Rejected on size	PASS — rejected, size exceeded
TC-14	Five rapid requests; limit 3 per minute	First 3 allowed; remainder blocked	PASS — 1–3 allowed, 4–5 blocked
TC-15	Valid multimodal submission	Accepted and logged	PASS — audit entry written

```

terminal - security & rate-limit tests

STEP 10: SECURITY / INPUT VALIDATION TEST CASES
=====
valid_text      -> PASS(valid)  errors=[]
empty_text      -> REJECTED    errors=['Text query is empty.']
oversized_text  -> REJECTED    errors=['Text query exceeds max length 2000 chars.']
prompt_injection -> REJECTED    errors=['Potential prompt-injection pattern detected: 'reveal (your|the) (prompt|api key|credentials)']
valid_image     -> PASS(valid)  errors=[]
bad_ext_image   -> REJECTED    errors=['Disallowed image extension '.exe'.']
oversized_image -> REJECTED    errors=['Image exceeds max size 8388608 bytes.']
valid_audio     -> PASS(valid)  errors=[]
=====

STEP 11: RATE LIMITER TEST (max 3 requests / 60s window)
=====
Request 1: ALLOWED
Request 2: ALLOWED
Request 3: ALLOWED
Request 4: BLOCKED (rate limit exceeded)
Request 5: BLOCKED (rate limit exceeded)
=====

```

Figure 9.1 — Security, validation and rate-limiter test evidence.

10. Execution Evidence and Model Card

The demonstration script builds the index, prints corpus statistics, runs retrieval queries and agent traces, executes three fusion cases and exercises validation and rate-limit cases. The following evidence images are carried over from the reference document.

```

terminal - python3 src/run_demo.py

STEP 1: BUILD RAG INDEX (chunking + TF-IDF/SVD embedding + FAISS)
=====
Documents loaded : 5
- MAN-CP-204      Centrifugal Pump Model CP-204 - Operation and Maintenance Manual (Exce
- SOP-MECH-014    Standard Operating Procedure - Rotating Equipment Bearing Inspection (
- LOG-INC-2026    Incident Log Extract - Rotating Equipment, Plant Line 3 (2026)
- REF-SCHED-07    Preventive Maintenance Interval Reference - Rotating Equipment (Refere
- POL-AI-003      Plant Digital Systems - AI-Assisted Diagnostics Governance Policy (Exc
Total chunks      : 22 (chunk_size=60 words, overlap=15 words)
Embedding dim     : 128 (TF-IDF -> TruncatedSVD)
Index build time  : 0.0102 s
=====

STEP 2: RETRIEVAL - SAMPLE QUERY 1
=====
Query: Non-drive-end bearing on the pump is making a high pitched whining noise and vibration is rising
Latency: 0.64 ms
Top-3 retrieved chunks:
[1] score=0.7042 MAN-CP-204::chunk4
    4.5 mm/s. Use only manufacturer-approved bearing part number BRG-6205-2RS. Do not reuse removed bearings. 7.4 Common Fault Codes - C
[2] score=0.6945 LOG-INC-2026::chunk0
    DOC_ID: LOG-INC-2026 TITLE: Incident Log Extract - Rotating Equipment, Plant Line 3 (2026) INCIDENT 2026-0142 Equipment: CP-204 Feed
[3] score=0.2698 MAN-CP-204::chunk2
    and parallel offset are within 0.05 mm using a dial indicator or laser alignment tool. 2. Bearing condition - listen for grinding or
=====

```

Figure 10.1 — RAG index build and retrieval console output.

```

terminal - retrieval quality summary

STEP 14: RETRIEVAL QUALITY SUMMARY TABLE (manual relevance judgement)
=====
Query                                     Top-1 chunk                               Top-1 score Relevant?(manual)
Non-drive-end bearing on the pump is making  MAN-CP-204::chunk4                       0.7042      Yes
What should I check if there is crackling no LOG-INC-2026::chunk2                     0.8357      Yes

DEMO RUN COMPLETE.

```

Figure 10.2 — Retrieval-quality summary generated by the evaluation script.

10.1 Model card

Field	Value
System name	IndustroSense AI
Version	0.2.0 (lab prototype)
Intended use	Assisting field technicians in diagnosing CP-2xx centrifugal feedwater pump faults from text, image and voice reports, grounded in plant manuals, SOPs and incident logs.
Out-of-scope use	Equipment outside CP-2xx; substitution for certified engineer sign-off on safety-critical work; autonomous plant control.
Known limitations	Local TF-IDF/SVD retrieval is weaker for paraphrase-heavy queries; vision and speech channels are simulated; injection defence is pattern-based and not red-teamed.
Human oversight	Any output below 0.60 confidence is labelled as requiring human review and queued for a certified engineer.
Data handling	Uploaded media is retained only for the defined window, excluded from training and not written to logs beyond hash and byte count.

11. Results, Analysis and Validation

11.1 Retrieval quality

Query	Top-1 chunk	Score	Relevant
Whining bearing, rising vibration	MAN-CP-204 chunk 4	0.71	Yes
Crackling near suction, pressure drop	LOG-INC-2026 chunk 2	0.83	Yes
Restart sequence after inspection	SOP-PMP-011 chunk 1	0.68	Yes

Top-1 precision was three of three on this evaluation set. The figure is indicative only at 22 chunks; a production evaluation should report precision@k, recall@k and mean reciprocal rank against a labelled technician query set.

11.2 Fusion and confidence behaviour

Confidence tracked evidence rather than verbosity: the three-channel cases scored 0.84 and 0.79, while the single-channel case with a weak match fell to 0.52 and was flagged. This is the intended governance behaviour — the score must fall when evidence thins.

11.3 Schedule computation

A last service at 15000 hours plus an 18000-hour bearing interval places the next service at 33000 hours. A current reading of 34500 hours is therefore overdue by 1500 hours. The agent matched the hand calculation exactly and correctly asked for clarification when operands were missing.

11.4 Latency and security

Operation	Measured
Index build: 5 documents to 22 chunks	0.011 s
Single retrieval, top-3 over 22 chunks	0.5–0.7 ms
Full three-channel fusion with simulated channels	under 0.1 s
Security classification accuracy	9/9 correct classifications
Rate limiter	3 admitted; 4th and 5th blocked

12. Analysis, Trade-offs and Justification

Decision	Alternative	Reason for final choice
Flat inner-product index	HNSW / IVF	Exact retrieval is fast enough and safer at 22 chunks; HNSW is the scale-up path.
Late text-level fusion	Early embedding-level fusion	Per-channel attribution, auditability and graceful degradation outweigh possible cross-modal gains.
Deterministic schedule calculator	Generated numeric answer	A calculation is reproducible and prevents safety-relevant numerical hallucination.
Human-review threshold at 0.60	Always answer	Uncertainty is transferred to a qualified reviewer rather than hidden in fluent text.

The dominant risk in this application is not a slow answer but a confident wrong one acting on plant equipment. Each trade-off therefore resolves in favour of exactness, inspectability, deterministic computation and refusal over a plausible guess.

13. Responsible Computing, Ethics and Sustainability

Sustainability: retrieving only the top few chunks keeps token generation, hosted-model cost and energy use proportionate to the question. **Accessibility:** accepting photographs and speech helps technicians working in gloves, on a noisy floor, or with varying typing speed and language fluency. **Ethics:** plant imagery and voice recordings are treated as operational data; retention is bounded, training use is excluded and low-confidence outputs are escalated. SDG 4 is supported through applied learning, SDG 9 through industrial maintenance innovation, and SDG 12 through efficient retrieval and more accurate maintenance scheduling.

14. Conclusion, Limitations and Possible Improvements

14.1 Conclusion

The three sub-problems were addressed in one integrated system. Retrieval is grounded and cited, the agent chooses between lookup, calculation and clarification, three modalities are fused into a traceable verdict, and governance controls refuse crafted abuse cases while flagging weakly supported answers.

14.2 Limitations

- The local TF-IDF+SVD embedding is weaker on synonym-rich paraphrases than a true semantic model.
- Vision and speech modules are deterministic stubs, so fusion results demonstrate pipeline mechanics rather than production perception accuracy.

- The evaluation set is small and establishes correctness of mechanism rather than statistical performance.
- The pattern-based injection defence has not been adversarially red-teamed.
- Authentication and secrets management are designed but not fully implemented because the prototype makes no external network calls.

14.3 Possible improvements

- Replace offline embeddings with a hosted or open-source dense model and evaluate against a larger labelled query set.
- Replace simulated vision and speech functions with real VLM and ASR services behind the same contracts.
- Migrate to HNSW when the corpus grows beyond a few thousand chunks.
- Add hybrid keyword-and-dense retrieval so exact fault codes are never missed.
- Record reviewer outcomes to recalibrate confidence thresholds and measure disagreement.

15. Individual Reflection

Building IndustroSense AI required integrating three course threads — RAG with vector databases, multimodal generative AI, and responsible-AI/security governance — that were taught as separate units. My biggest design lesson was how tightly those threads depend on each other in a real system. The multimodal fusion strategy was not chosen only for modelling performance; the governance requirement for per-modality auditability made late, text-level fusion the more responsible choice. Similarly, the agent's decision to ask for missing maintenance inputs rather than guess a number showed me that agent design and responsible AI are the same design conversation when outputs affect physical equipment.

The main technical challenge was the development environment. There was no network route to hosted embedding, vision or speech APIs, so I substituted a genuinely offline TF-IDF plus Truncated-SVD pipeline and clearly documented simulated vision and speech functions. I learned that an honestly labelled offline substitute is more valuable than an impressive but unrunnable claim. Getting the FAISS index, ReAct-style router and confidence-gated fusion to run end-to-end was especially useful because it exposed edge cases that a theoretical design would have missed, including schedule queries with missing numeric fields.

The most important professional lesson was that a good AI system must know when not to answer. The 0.52 single-channel case being routed to human review made the governance policy feel like engineering rather than paperwork. I also learned to value simple, replaceable interfaces: because the simulated channels use the same signatures as real APIs, future upgrades can be made without rewriting retrieval or fusion.

This assignment connects most directly to SDG 9 through its focus on industrial maintenance infrastructure and to SDG 12 through efficient retrieval and better maintenance scheduling. It also maps clearly to CO3, CO4 and CO5. If I continued the work, I would first build a larger labelled evaluation set, then replace the local embedding model and conduct formal adversarial red-teaming of the injection filter. Those steps would test whether the conservative prototype remains reliable under realistic technician language and deliberate misuse.

Overall, the project changed my view of generative AI from a system that produces fluent answers into a system that must expose evidence, quantify uncertainty and preserve human responsibility. That shift is the main learning outcome I will carry into future AI projects.

16. References

1. T. Chakraborty, Introduction to Large Language Models: Generative AI for Text, Wiley, 2024.
2. P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” NeurIPS, 2020.
3. J. Johnson, M. Douze and H. Jégou, “Billion-scale similarity search with GPUs,” IEEE Transactions on Big Data, 2019.
4. Y. Malkov and D. Yashunin, “Efficient and Robust Approximate Nearest Neighbor Search using Hierarchical Navigable Small World Graphs,” IEEE TPAMI, 2020.
5. M. Mitchell et al., “Model Cards for Model Reporting,” ACM FAccT, 2019.
6. National Institute of Standards and Technology, AI Risk Management Framework 1.0, 2023.
7. OWASP Foundation, Top 10 for Large Language Model Applications, 2025.
8. TfidfVectorizer and TruncatedSVD documentation, scikit-learn developers, 2026.
9. Streamlit Inc., “Streamlit documentation,” <https://docs.streamlit.io>, accessed 2026.
10. ISO 10816, “Mechanical vibration — Evaluation of machine vibration by measurements on non-rotating parts.”
11. Dataset: five synthetically authored sample documents created for this assignment to represent a plant technical corpus.
12. Software/tools: Python 3.12, faiss-cpu, scikit-learn, Streamlit, Pillow and cairosvg.
13. AI-assisted tools: a general-purpose AI assistant supported drafting and code review; all design decisions, testing and analysis are the author’s own.

Appendix — Common Assessment Rubric

Assessment criterion	Marks
Problem understanding and formulation	10
Application of course / domain knowledge	20
Solution methodology / design / implementation	20
Testing, results and validation	15
Analysis, trade-offs and justification	10
Responsible computing, ethics and sustainability	10
Documentation, presentation and reflection	15
Total	100